



西南财经大学

SOUTHWESTERN UNIVERSITY OF FINANCE AND ECONOMICS

Algorithm Design & Analysis

Lecture 4

SWUFE

经世济民，孜孜以求

A dramatic landscape featuring a dark, stormy sky with heavy, textured clouds. A path leads from the bottom left towards the horizon, flanked by green fields and some trees. The sun is setting or rising, creating a bright glow on the horizon and illuminating the clouds from below. The overall mood is intense and powerful.

Divide *and* Conquer

改进方法1

降低分解数量

改进方法2

增加预处理

芯片测试

快速排序

幂乘算法

分治算法的一般性描述
分治算法的分析方法

分治算法的设计思想
例子：二分检索、二分归并



一、分治策略设计思想

二、分治策略的描述与分析方法

三、分治策略简单例子

芯片测试

快速排序

幂乘算法及应用

四、改进分治算法





分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

分治策略的基本思想

分治策略(Divide and Conquer)

1. **分解**：将原始问题**划分**或者归结为规模**较小**的子问题
2. **解决**：**递归**或**迭代**求解每个子问题
3. **合并**：将子问题的解**综合**得到原问题的解

Note:

1. 子问题与原问题性质完全一样
2. 子问题之间可彼此独立地求解
3. 递归停止时子问题可直接求解



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

二分检索

算法 BinarySearch(T, x)

输入：排好序数组 T ，数 x

输出： j // 如果 x 在 T 中， j 为下标；否则为 0

1. $l \leftarrow 1; r \leftarrow n$

2. while $l \leq r$ do

3. $m \leftarrow \lfloor (l+r)/2 \rfloor$

4. if $T[m] = x$ then return m // x 恰好等于中位元素

5. else if $T[m] > x$ then $r \leftarrow m-1$

6. else $l \leftarrow m+1$

7. return 0



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

二分检索算法设计思想

- **分解**：通过 x 与中位数的比较，将原问题归结为规模减半的子问题，如果 x 小于中位数，则子问题由小于 x 的数构成，否则子问题由大于 x 的数构成。
- **解决**：对子问题进行二分检索
- **合并**：当子问题规模为1时，直接比较 x 与 $T[m]$ ，若相等则返回 m ，否则返回0



能否写成递归实现呢？

迭代：旧值推新值 递归：调用自身，逐层返回



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

二分检索时间复杂度

二分检索问题最坏情况下时间复杂度

$$W(n) = W(\lfloor n/2 \rfloor) + 1$$

$$W(1) = 1$$

可以解出

$$W(n) = \lfloor \log n \rfloor + 1$$



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

二分归并排序算法

算法: MergeSort(A, p, r)

输入: 数组 $A[p \dots r]$

输出: 按递增顺序排序的数组 A

1. if $p < r$

2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$

3. MergeSort(A, p, q)

4. MergeSort($A, q+1, r$)

5. Merge(A, p, q, r)

对半划分

子问题1

子问题2

综合求解



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

时间复杂度

小结

二分归并排序设计思想

- **分解**：将原问题归结为规模为 $n/2$ 的2个子问题.
- **解决**：将原问题归结为规模为 $n/4$ 的4个子问题.继续…，当子问题规模为1时，划分结束.
- **合并**：从规模1到 $n/2$ ，陆续归并被排好序的两个子数组.每归并一次，数组规模扩大一倍，直到恢复到原始数组



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

二分归并排序时间复杂度

假设n为2的幂，二分归并排序最坏情况下时间复杂度

$$\begin{cases} W(n) = 2W\left(\frac{n}{2}\right) + n - 1 \\ W(1) = 0 \end{cases}$$

可以解出

$$W(n) = n \log n - n + 1.$$



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

Hanoi塔的递归算法

算法 $\text{Hanoi}(A, C, n)$ // 将A的 n 个盘子按要求移到C

1. if $n=1$ then move (A, C) // 将A的1个盘子移到C

2. else $\text{Hanoi}(A, B, n-1)$

3. move (A, C)

4. $\text{Hanoi}(B, C, n-1)$

设 n 个盘子的移动次数为 $T(n)$

$$T(n) = 2T(n-1) + 1,$$

$$T(1) = 1,$$

$$T(n) = 2^n - 1$$

实际也是分治策略，只
不过每次子问题只减少1



分治策略

基本思想

二分检索

伪码

设计思想

时间复杂度

二分归并

伪码

设计思想

时间复杂度

汉诺塔

伪码

设计思想

小结

Hanoi塔的设计思想

- **分解**：将原问题归结为规模为 $n-1$ 的2个子问题.
- **解决**：将原问题归结为规模为 $n-2$ 的4个子问题.继续…，当子问题规模为1时，解决子问题过程结束.
- **合并**：从规模1到 $n-1$ ，陆续组合两个子问题的解.直到规模为 n .



小结

通过几个例子展示分治算法的特点：

- 将原问题规约为规模小的子问题，
子问题与原问题具有相同的性质.
- 子问题规模足够小时可直接求解.
- 算法可以递归也可以迭代实现.
- 算法的分析方法：递推方程



一、分治策略设计思想

二、分治策略的描述与分析方法

三、分治策略简单例子

芯片测试

快速排序

幂乘算法及应用

四、改进分治算法





描述与分析

描述方法

设计要点

时间分析方法

常见递推方程

两类方程

方程求解

方程2的解

小结

分治算法的一般性描述

分治算法 Divide-and-Conquer(P)

1. if $|P| \leq c$ then $S(P)$.

2. divide P into $P_1, P_2, \dots, P_k$

分解

3. for $i \leftarrow 1$ to k

4. $y_i \leftarrow \text{Divide-and-Conquer}(P_i)$

求解

5. Return Merge($y_1, y_2, \dots, y_k$)

综合解



描述与分析

描述方法

设计要点

时间分析方法

常见递推方程

两类方程

方程求解

方程2的解

小结

设计要点

- 原问题可以划分或者规约为规模较小的子问题
 - 子问题与原问题**具有相同的性质**
 - 子问题的求解**彼此独立**
 - 划分时子问题的规模**尽可能均衡**
- 子问题规模足够小时可以直接求解
- 子问题的解综合得到原问题的解
- 算法实现：递归或迭代



描述与分析

描述方法

设计要点

时间分析方法

常见递推方程

两类方程

方程求解

方程2的解

小结

分治算法时间分析

算法时间复杂度的递推方程

$$\begin{cases} W(n) = W(|P_1|) + W(|P_2|) + \dots + W(|P_k|) + f(n) \\ W(c) = C \end{cases}$$

- $P_1, P_2, \dots, P_k$ 为划分后产生的子问题
- $f(n)$ 为划分子问题以及将子问题的解综合得到原问题解的总工作量
- 规模为 c 的最小子问题的工作量为 C



描述与分析

描述方法

设计要点

时间分析方法

常见递推方程

两类方程

方程求解

方程2的解

小结

两类常见的递推方程

$$T(n) = \sum_{i=1}^k a_i T(n-1) + f(n)$$

$$T(n) = aT\left(\frac{n}{b}\right) + d(n)$$

例子：

Hanoi塔, $W(n) = 2W(n-1) + 1$

二分检索, $W(n) = W(n/2) + 1$

归并排序, $W(n) = 2W(n/2) + n - 1$



描述与分析

描述方法

设计要点

时间分析方法

常见递推方程

两类方程

方程求解

方程2的解

小结

递推方程的求解

$$T(n) = \sum_{i=1}^k a_i T(n-1) + f(n) \quad \text{方程1}$$

$$T(n) = aT\left(\frac{n}{b}\right) + d(n) \quad \text{方程2}$$

求解方法：

第一类方程：迭代法、换元法、递归树、尝试法

第二类方程：迭代法、递归树、主定理



描述与分析

描述方法

设计要点

时间分析方法

常见递推方程

两类方程

方程求解

方程2的解

小结

方程2的解

方程 $T(n) = aT(n/b) + d(n)$

$d(n)$ 为常数

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & a \neq 1 \\ \Theta(\log n) & a = 1 \end{cases}$$

$d(n) = cn$

$$T(n) = \begin{cases} \Theta(n) & a < b \\ \Theta(n \log n) & a = b \\ \Theta(n^{\log_b a}) & a > b \end{cases}$$



小结

- 分治算法的一般描述

划分或归约为彼此独立的子问题

分别求解每个子问题

给出递归或迭代计算的终止条件

如何由子问题的解得到原问题解

- 分治算法的分析方法

求解时间复杂度的递推方程

常用的递推方程的解



一、分治策略设计思想

二、分治策略的描述与分析方法

三、分治策略简单例子

芯片测试

快速排序

幂乘算法及应用

四、改进分治算法



● 芯片测试

● 测试介绍

● 结果分析

● 问题定义

● 判断芯片好坏
奇数情况

偶数情况

● 蛮力算法

● 采用分治思想

● 算法正确性

偶数情况

奇数情况

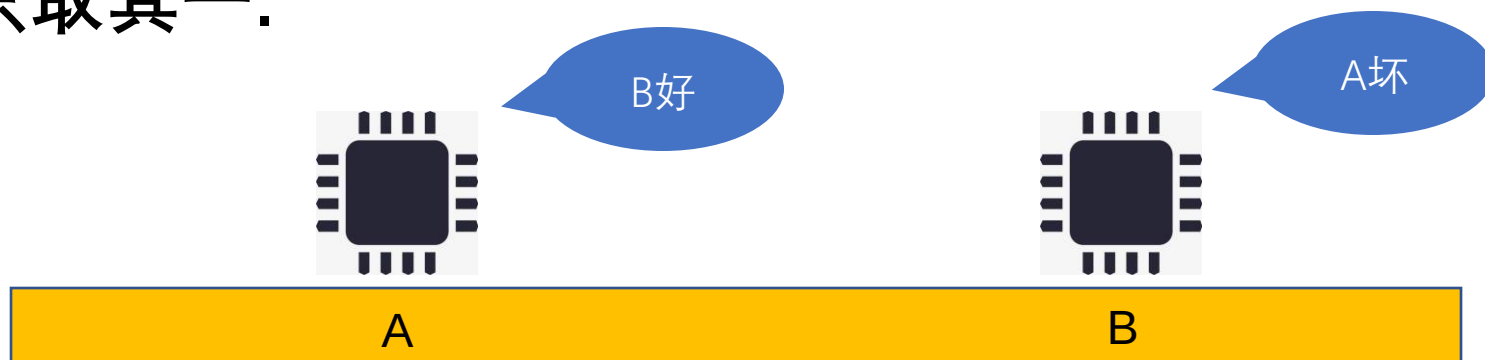
● 伪码描述

● 时间复杂度

● 小结

芯片测试例子

测试方法：将2片芯片（A和B）置于测试台上，互相进行测试，测试报告是“好”或者“坏”，只取其一。



假设：好芯片的报告一定是正确的，坏芯片的报告是不确定的（可能会出错）



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏 奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性

偶数情况

奇数情况

伪码描述

时间复杂度

小结

测试结果分析

A 报告	B 报告	结论
B是好的	A是好的	A,B 都好或 A,B 都坏
B是好的	A是坏的	至少一片是坏的
B是坏的	A是好的	至少一片是坏的
B是坏的	A是坏的	至少一片是坏的



● 芯片测试

● 测试介绍

● 结果分析

● 问题定义

● 判断芯片好坏
奇数情况

偶数情况

● 蛮力算法

● 采用分治思想

● 算法正确性

偶数情况

奇数情况

● 伪码描述

● 时间复杂度

● 小结

问题定义

输入：

n 片芯片，其中**好芯片至少比坏芯片多1片**。

问题：

设计一种测试方法，通过测试从 n 片芯片中挑出一片好芯片

要求：

使用最少的测试次数。



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏
奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性

偶数情况

奇数情况

伪码描述

时间复杂度

小结

判断芯片A的好坏(奇数)

好芯片至少比
坏芯片多1片

问题：给定芯片A，判定A的好坏

方法：用其他 $n-1$ 片芯片对A测试.

例子： $n=7$ ，好芯片 ≥ 4 .

A好，6个报告中至少3个报“好”

A坏，6个报告中至少4个报“坏”

n 是奇数：好芯片数 $\geq (n+1)/2$.

A好，至少 $(n-1)/2$ 个报“好”

A坏，至少 $(n+1)/2$ 个报“坏”

结论：至少一半报“好”，A是好芯片，
超过一半报“坏”，A是坏芯片.



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏

奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性

偶数情况

奇数情况

伪码描述

时间复杂度

小结

判断芯片A的好坏(偶数)

好芯片至少比
坏芯片多1片

例子： $n=8$ ，好芯片 ≥ 5 .

A好，7个报告中至少4个报“好”

A坏，7个报告中至少5个报“坏”

n是偶数：好芯片数 $\geq n/2 + 1$.

A好，至少 $n/2$ 个报“好”

A坏，至少 $n/2 + 1$ 个报“坏”

结论： $n-1$ 份报告中，
至少一半报“好”，A是好芯片，
超过一半报“坏”，A是坏芯片.



● 芯片测试

● 测试介绍

● 结果分析

● 问题定义

● 判断芯片好坏
奇数情况

偶数情况

● 蛮力算法

● 采用分治思想

● 算法正确性

偶数情况

奇数情况

● 伪码描述

● 时间复杂度

● 小结

蛮力算法

测试方法：任取1片测试，如果是好芯片，测试结束；如果是坏芯片，抛弃，再从剩下芯片中取出1片测试，直到得到1片好芯片。

时间估计：

第1片坏芯片，最多测试 $n-2$ 次，

第2片坏芯片，最多测试 $n-3$ 次，

...

总计 $\Theta(n^2)$



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏

奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性

偶数情况

奇数情况

伪码描述

时间复杂度

小结

分治算法设计思想

好芯片至少比
坏芯片多1片

假设 n 为偶数，将 n 片芯片两两一组做测试淘汰，剩下芯片构成子问题，进入下一轮分组淘汰。

淘汰规则：

“好，好” $\Rightarrow$ 任留1片，进入下一轮

其他情况 $\Rightarrow$ 全部抛弃 (如何确保子问题与原问题一致?)

递归截至条件： $n \leq 3$

3片芯片，1次测试可得到好芯片

1或2片芯片，不再需要测试。



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏
奇数情况
偶数情况

蛮力算法

采用分治思想

算法正确性
偶数情况
奇数情况

伪码描述

时间复杂度

小结

分治算法的正确性(偶数)

命题1: 当 n 是偶数时, 在上述淘汰规则下, 经过一轮淘汰, 剩下的好芯片比坏芯片至少多1片. 如何保证好芯片多?

证: 设 A , B 都好的芯片 i 组, A 与 B 一好一坏 j 组, A 与 B 都坏的 k 组, 淘汰后好芯片至少 i 片, 坏芯片至多 k 片.

$$2i + 2j + 2k = n$$

初始芯片总数

$$2i + j > 2k + j \Rightarrow i > k$$

初始好芯片多于坏芯片



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏
奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性
偶数情况
奇数情况

伪码描述

时间复杂度

小结

分治算法的正确性(奇数)

当 n 是奇数时，可能出问题

输入：好 好 好 好 坏 坏 坏

分组：好 好 好 好 坏 坏 坏

淘汰：好 好 坏 坏

伪装成
好芯片

处理办法：当 n 为奇数时，增加一轮对轮空芯片的单独测试。

如果该芯片为好芯片，算法结束；
如果是坏芯片，则淘汰该芯片。



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏
奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性

偶数情况

奇数情况

伪码描述

时间复杂度

小结

伪码描述

Test(n)

```
1 .  $k \leftarrow n$ 
2 . while  $k > 3$  do
3 .   将芯片分成  $\lfloor k/2 \rfloor$  组 // 如有轮空芯片, 特殊处理
4 .   for  $i = 1$  to  $\lfloor k/2 \rfloor$  do
       if 2片好, 则任取1片留下
       else 2片同时丢掉
5 .    $k \leftarrow$  剩下的芯片数
6 .   if  $k = 3$ 
       then 任取2片芯片测试
            if 1好1坏 then 取没测的芯片
            else 任取1片被测芯片
7 .   if  $k = 2$  or  $1$  then 任取1片
```

分组
淘汰

递归
结束



芯片测试

测试介绍

结果分析

问题定义

判断芯片好坏 奇数情况

偶数情况

蛮力算法

采用分治思想

算法正确性

偶数情况

奇数情况

伪码描述

时间复杂度

小结

时间复杂度分析

设输入规模为 n

每轮淘汰后，芯片数至少减半

测试次数(含轮空处理): $O(n)$ (精确值为 $\lfloor n/2 \rfloor + \lfloor n/2 \rfloor$)

时间复杂度是:

$$\begin{cases} W(n) = W\left(\frac{n}{2}\right) + O(n) & n > 3 \\ W(n) = 1 & n \leq 3 \end{cases} \Rightarrow W(n) = O(n)$$

解得

蛮力算法

$$W(n) = O(n^2)$$

优于
蛮力



小结

- 芯片测试得分治算法

如何保证子问题与原问题性质相同：增加额外处理

额外处理的工作量不改变函数的阶

时间复杂度为 $O(n)$



一、分治策略设计思想

二、分治策略的描述与分析方法

三、分治策略简单例子

芯片测试

快速排序

幂乘算法及应用

四、改进分治算法





快速排序 Quick Sort

4 6 2 3 1 5 7 8

霍尔 (Sir Charles Antony Richard Hoare) 英国计算机科学家
26岁发明**快速排序**算法的发明者。

1980年他因“对程序设计语言的定义和设计方面的基础性贡献”而获得**图灵奖**。





快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

快速排序-基本思想

- 用首元素 x 作为划分标准，将输入数组 A 划分成不超过 x 的元素构成的数组 A_L ，大于 x 的元素构成的数组 A_R ，其中 A_L, A_R 从左到右存放在数组 A 的位置。
- 递归地对子问题 A_L, A_R 进行排序，直到子问题规模为1时停止。



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

伪码

算法 Quicksort(A, p, r)

输入：数组 $A[p..r]$

输出：排好序的数组 A

1. if $p < r$
2. then $q \leftarrow \text{Partition}(A, p, r)$
3. $A[p] \leftrightarrow A[q]$
4. Quicksort($A, p, q-1$)
5. Quicksort($A, q+1, r$)

初始置 $p=1, r=n$ ，然后调用上述算法



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

划分过程-Partition(A, p, r)

输入：数组 $A[p, r]$

输出： j ， A 的首元素在排好序的数组中的位置

1. $x \leftarrow A[p]$

2. $i \leftarrow p$

3. $j \leftarrow r+1$

4. while true do

5. repeat $j \leftarrow j-1$

6. until $A[j] \leq x$ //从后往前找不超过首元素的

7. repeat $i \leftarrow i+1$

8. until $A[i] > x$ //从前往后找比首元素大的

9. if $i < j$

10. then $A[i] \leftrightarrow A[j]$

11. else return j

划分实例

27 99 0 8 13 64 86 16 7 10 88 25 90

i *j*

27 25 0 8 13 64 86 16 7 10 88 99 90

i j

27 25 0 8 13 10 86 16 7 64 88 99 90

i j

27 25 0 8 13 10 7 16 86 64 88 99 90

j i

16 25 0 8 13 10 7 27 86 64 88 99 90



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

时间复杂度

最坏情况

$$W(n) = W(n-1) + n - 1$$

$$W(1) = 0$$

$$W(n) = \frac{1}{2}n(n-1) = \Theta(n^2)$$

另一个子问题没有了

最好划分

$$T(n) = 2T\left(\frac{n}{2}\right) + n - 1$$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

均衡划分的时间复杂度

均衡划分：子问题的规模比不变，比如：1:9

$$T(n) = T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + O(n)$$
$$T(1) = 0$$

为何是n

根据递归树，时间复杂度

$$T(n) = \Theta(n \log n)$$



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

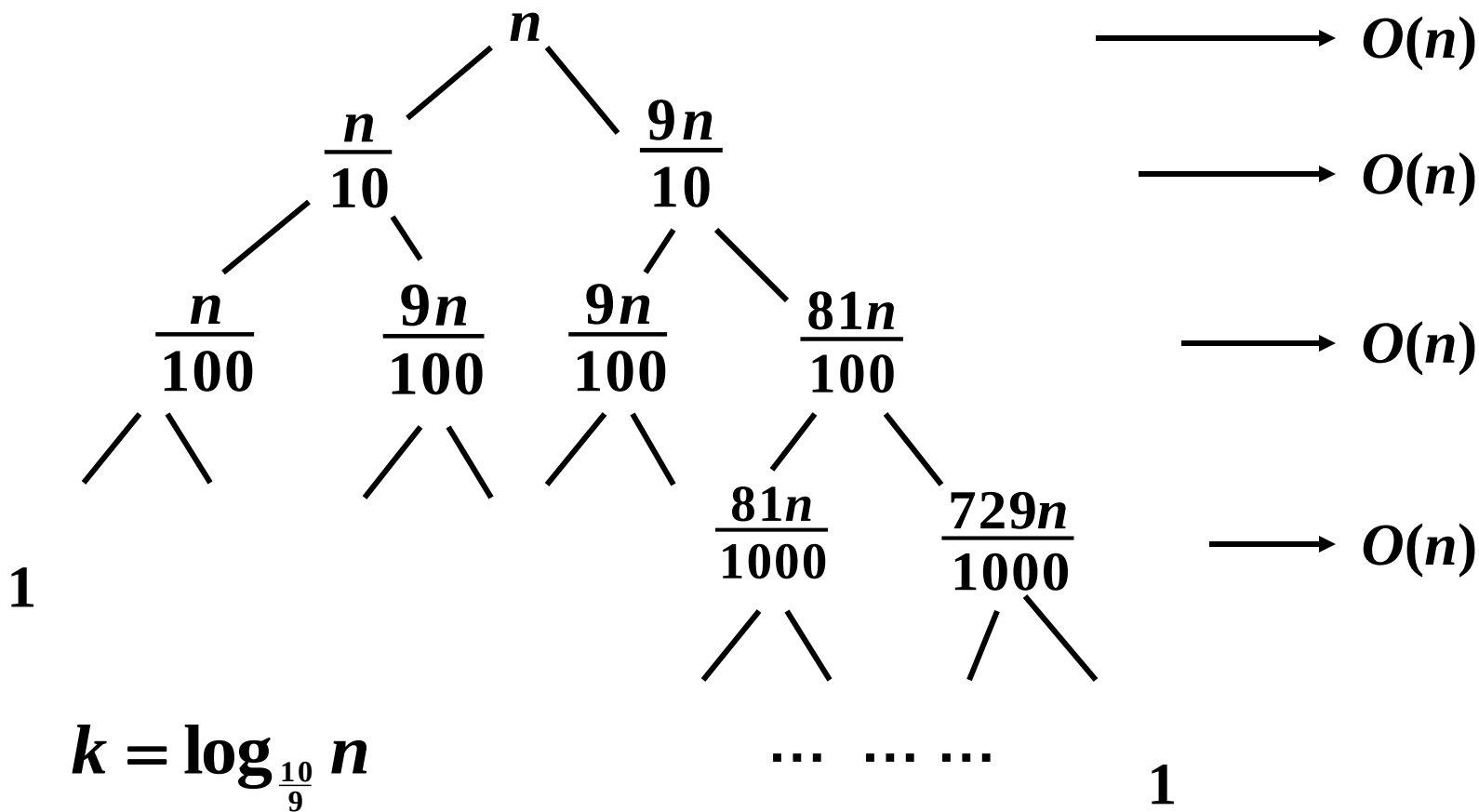
平均复杂度

分析

结果

小结

均衡划分递归树



$$T(n) = O(n \log n)$$



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

平均时间复杂度分析

首元素排好序后处在1, 2..., n
各种情况概率都是1/n

首元素在位置1: $T(0), T(n-1)$

首元素在位置2: $T(1), T(n-2)$

...

首元素在位置n-1: $T(n-2), T(1)$

首元素在位置n: $T(n-1), T(0)$

子问题工作量 $2[T(1)+T(2)+...T(n-1)]$

划分工作量 $n-1$



快速排序

伪码

划分过程

划分实例

时间复杂度

非均衡划分

均衡划分

均衡划分递归树

平均复杂度

分析

结果

小结

平均时间复杂度

$$T(n) = \frac{1}{n} \sum_{k=1}^{n-1} (T(k) + T(n-k)) + n - 1$$

$$T(n) = \frac{2}{n} \sum_{k=1}^{n-1} T(k) + n - 1 \quad T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

利用差消法求解

首元素排划分后每个位置概率相等



小结

- 快速排序算法
 - 分治策略
 - 子问题划分是由首元素决定的
 - 最坏情况下时间 $O(n^2)$
 - 平均情况下时间为 $O(n\log n)$



一、分治策略设计思想

二、分治策略的描述与分析方法

三、分治策略简单例子

芯片测试

快速排序

幂乘算法及应用

四、改进分治算法





幂乘问题

问题定义

分治-划分

算法分析

幂乘应用

Fibonacci 数

Fib...数性质

Fib...数性质2

采用分治思想

小结

幂乘问题

输入： a 为给定实数， n 为自然数

输出： a^n

传统算法： 顺序相乘

$$a^n = (\dots (((a \times a)a)a) \dots)a$$

乘法次数： $\Theta(n)$



● 幂乘问题

● 问题定义

● 分治-划分

● 算法分析

● 幂乘应用

Fibonacci 数

● Fib...数性质

● Fib...数性质2

● 采用分治思想

● 小结

分治算法---划分

n为偶数 $\underbrace{a \dots a}_{n/2} \mid \underbrace{a \dots a}_{n/2}$

n为奇数 $\underbrace{a \dots a}_{(n-1)/2} \mid \underbrace{a \dots a}_{(n-1)/2} \mid a$

最后计算方式为

$$a^n = \begin{cases} a^{n/2} \times a^{n/2}, & n \text{ is even} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a, & n \text{ is odd} \end{cases}$$



● 幂乘问题

● 问题定义

● 分治-划分

● 算法分析

● 幂乘应用

Fibonacci 数

● Fib...数性质

● Fib...数性质2

● 采用分治思想

● 小结

分治算法分析

以乘法作为基本运算

子问题规模：不超过 $n/2$

两个子规模近似 $n/2$ 的子问题完全一样，只需要计算1次

$$W(n) = W\left(\frac{n}{2}\right) + \Theta(1)$$

$$W(n) = \Theta(\log n)$$



● 幂乘问题

● 问题定义

● 分治-划分

● 算法分析

● 幂乘应用

Fibonacci 数

● Fib...数性质

● Fib...数性质2

● 采用分治思想

● 小结

幂乘算法的应用

Fibonacci 数

1, 1, 2, 3, 5, 8, 13, 21, ...

增加 $F_0=0$, 得到数列 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

问题：已知 $F_0, F_1 = 1$, 给定 n , 计算 F_n .

通常算法：从 $F_0, F_1, \dots$, 根据递推公式

$$F_n = F_{n-1} + F_{n-2}$$

陆续相加可得 F_n , 时间复杂度为 $\Theta(n)$



● 幂乘问题

● 问题定义

● 分治-划分

● 算法分析

● 幂乘应用

Fibonacci 数

● Fib...数性质

● Fib...数性质2

● 采用分治思想

● 小结

Fibonacci数的性质（分治算法）

定理1 设 $\{F_n\}$ 为 Fibonacci 数构成的数列，那么

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

归纳证明

$$n=1, \text{ 左边} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \text{右边}$$



● 幂乘问题

● 问题定义

● 分治-划分

● 算法分析

● 幂乘应用

Fibonacci 数

● Fib...数性质

● Fib...数性质2

● 采用分治思想

● 小结

Fibonacci数的性质 (续)

假设对任意正整数 n ,命题成立, 即

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

那么

$$\begin{aligned} \begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix} &= \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n+1} \end{aligned}$$

归纳假
设代入



● 幂乘问题

● 问题定义

● 分治-划分

● 算法分析

● 幂乘应用

Fibonacci 数

● Fib...数性质

● Fib...数性质2

● 采用分治思想

● 小结

算法(使用幂乘分治算法) Fibonacci

令矩阵 $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, 用乘法幂算法计算 M^n

时间复杂度:

- 矩阵乘法次数 $T(n) = \Theta(\log n)$
- 每次矩阵乘法需要做8次元素相乘
- 总计元素相乘次数为 $8 \times \log n = \Theta(\log n)$



小结

分治算法的例子---幂乘算法

幂乘算法的应用

计算Fibonacci数

通常算法 $O(n)$, 分治算法为 $O(\log n)$



一、分治策略设计思想

二、分治策略的描述与分析方法

三、分治策略简单例子

芯片测试

快速排序

幂乘算法及应用

四、改进分治算法





改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

改进分治算法的途径1

减少子问题个数

分治算法的复杂度方程：

$$W(n) = aW(n/b) + d(n)$$

a :子问题数, n/b :子问题规模

$d(n)$: 分解与综合工作量

当 a 较大, b 较小, $d(n)$ 不大时, 方程的解:

$$W(n) = \Theta(n^{\log_b a})$$

减少 a 是降低函数 $W(n)$ 的阶的途径.

利用子问题的依赖关系, 使某些子问题的解通过组合其他子问题的解而得到.(比如Fibonacci数)



改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

例子1：整数位乘问题

输入： X, Y 是 n 位二进制数， $n = 2^k$

输出： XY

普通乘法：需要 $O(n^2)$ 次位乘运算

用分治算法会不会好一些？如何设计分治算法？

简单分治法：令

X	A	B
Y	C	D

$$X = A * 2^{n/2} + B, Y = C * 2^{n/2} + D.$$

$$XY = \underline{AC} 2^n + (\underline{AD} + \underline{BC}) 2^{n/2} + \underline{BD}$$

拆分成了4个子问题

$$W(n) = 4W(n/2) + cn, W(1) = 1$$
$$W(n) = O(n^2)$$

并没有减少问题复杂度，原因？
(主定理可得)



改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

减少子问题个数

子问题的依赖关系：代数变换

$$XY = AC 2^n + \underline{(AD + BC) 2^{n/2}} + BD$$

$$\underline{AD + BC} = AD + BC - \underline{AC} - \underline{BD} + \underline{AC} + \underline{BD}$$

$$= (A - B)(D - C) + AC + BD$$

AC 、 BD 已经有计算结果，只需要计算 $(A - B)(D - C)$ 一个计算量

算法复杂度

$$W(n) = 3 W(n/2) + cn, \quad W(1) = 1$$

方程的解

$$W(n) = O(n^{\log 3}) = (n^{1.59})$$

原来的解 $O(n^2)$



改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

例2：矩阵相乘问题

输入： A, B 为 n 阶矩阵， $n = 2^k$

输出： $C = AB$

通常矩阵乘法：

C 中有 n^2 个元素

每个元素需要做 n 次乘法

以元素相乘为基本运算

$$W(n) = O(n^3)$$



改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

简单分治法

分治法 将矩阵分块，得

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

其中

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \quad C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21} \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

递推方程 $W(n) = 8W(n/2) + cn^2$

$$W(1) = 1$$

解 $W(n) = O(n^3)$

改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

Strassen 矩阵乘法

变换方法:

设计 $M_1, M_2, \dots, M_7$, 对应7个子问题

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

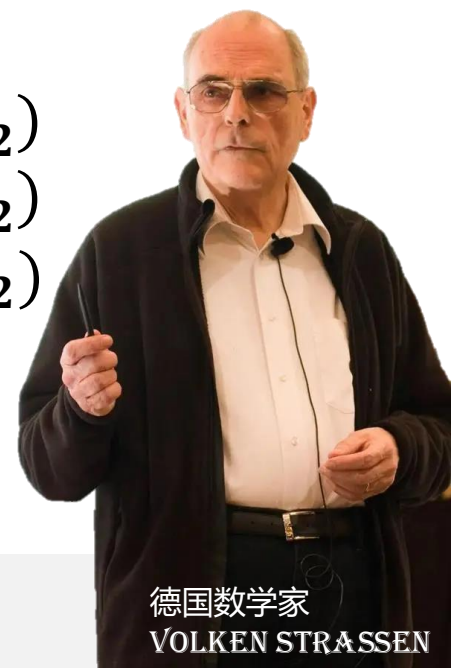
$$M_3 = (A_{21} + A_{22})B_{22}$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$



德国数学家
VOLKEN STRASSEN



改进分治1

途径1

整数位问题

设计分治算法

减少个数

矩阵相乘问题

简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

Strassen 矩阵乘法 (续)

利用中间矩阵，得到结果矩阵

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$

时间复杂度函数：

$$W(n) = 7W(n/2) + 18(n/2)^2$$
$$W(1) = 1$$

解 $W(n) = O(n^{\log 7}) = O(n^{2.8075})$

nature

MATRIX GAMES

Deep reinforcement learning opens route to faster algorithms for matrix multiplication



Protecting Peru
Can technology help Indigenous groups preserve the Amazon?

Invisible touch
How marine clouds are affected by aerosols emitted from shipping

Preferential practice
US universities favour prestige in faculty hiring and retention

DeepMind攻克50年数学难题！AlphaZero最快矩阵乘法算法登Nature

传统算法是用100次乘法对一个 4×5 乘以 5×5 的矩阵进行乘法，而这个数字在人类的聪明才智下被减少到80次，AlphaTensor已经找到了只用76次乘法就能完成同样操作的算法

ALPHATENSOR总共改进了70多个不同大小矩阵的计算方法



2020年10月，哈佛大学与MIT的两位研究者发表了一篇论文，他们创建了有史以来矩阵相乘的最快算法，相比于之前最快算法，计算复杂度下降了10万分之一。



改进分治1

途径1

整数位问题

设计分治算法
减少个数

矩阵相乘问题 简单分治

Strassen-1

Strassen-2

矩阵乘法应用

小结

矩阵乘法研究与应用

矩阵乘法问题的难度：

- Vassilevska Williams算法： $O(n^{2.37286})$, **最优上界**
- 目前为止最好的下界是： $\Omega(n^2)$

矩阵乘法的应用：

- 科学计算、深度学习、图像处理、数据挖掘等
- 回归、聚类、主成分分析、决策树等挖掘算法常涉及大规模矩阵运算



小结

- 适用于：子问题个数多，划分和综合工作量不太大，时间复杂度函数

$$W(n) = \Theta(n^{\log_b a})$$

- 利用子问题依赖关系，用某些子问题解的代数表达式表示另一些子问题的解，减少独立计算子问题个数.
- 综合解的工作量可能会增加，但增加的工作量不影响 $W(n)$ 的阶



改进分治2

平面最近点

划分实例

算法伪码

跨边界点处理

算法分析

增加预处理

实例分析

改进后复杂度

小结

改进分治算法的途径2

增加预处理

例子：平面最邻近点对算法

输入：平面点集 P 中有 n 个点, $n > 1$

输出： P 中的两个点，其距离最小

蛮力算法：

$C(n, 2)$ 个点对，计算最小距离， $O(n^2)$

分治策略： P 划分为大小相等的 P_L 和 P_R

1. 分别计算 P_L 、 P_R 中最近点对

2. 计算 P_L 与 P_R 中各一个点的最近点对

3. 上述情况下的最近点对是解

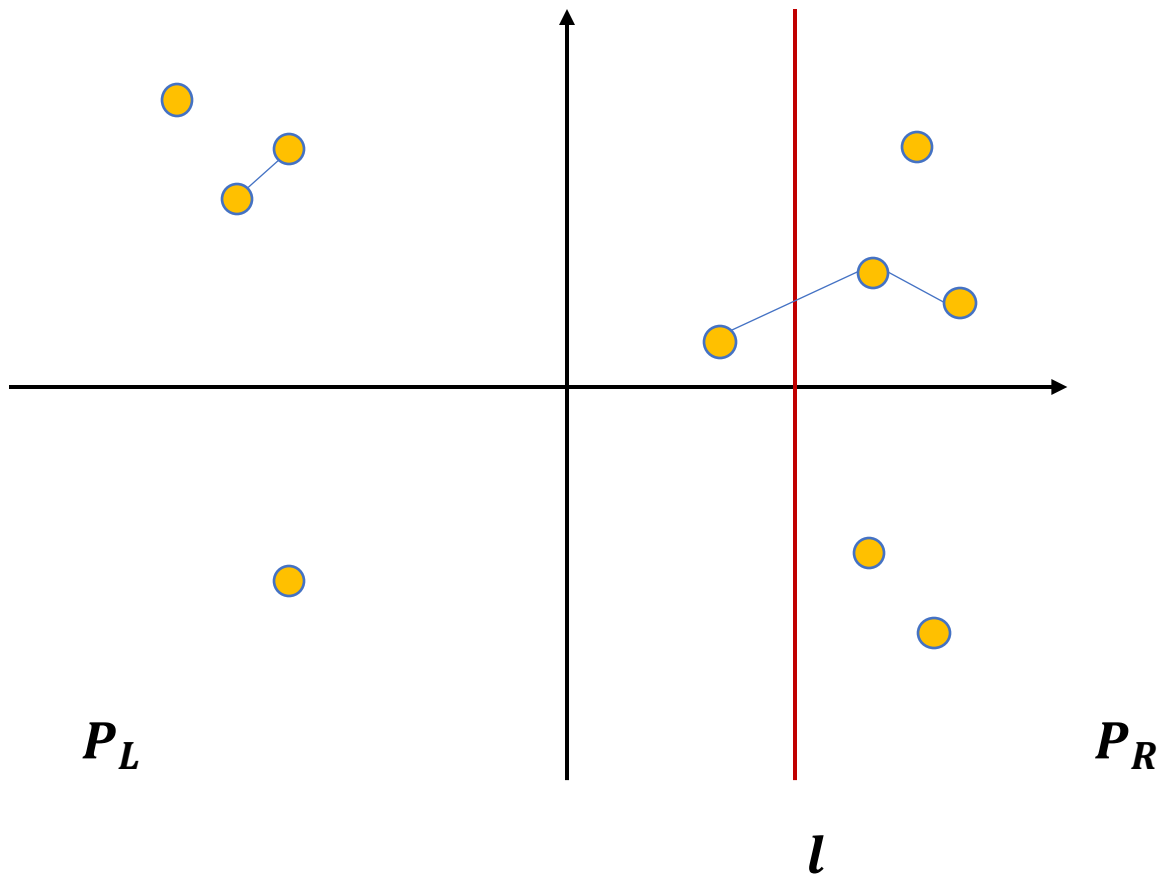


改进分治2

- 平面最近点
- 划分实例
- 算法伪码
- 跨边界点处理
- 算法分析
- 增加预处理
- 实例分析
- 改进后复杂度

小结

划分实例: $n = 10$





改进分治2

平面最近点
划分实例
算法伪码
跨边界点处理
算法分析
增加预处理
实例分析
改进后复杂度

小结

算法伪码

MinDistance(P, X, Y)

输入: n 个点的集合 P , X 和 Y 分别为横、纵坐标数组

输出: 最近的两个点及距离

1. 如果 $|P| \leq 3$, 直接计算其中的最小距离
2. 排序 X, Y
3. 做垂直线 l 将 P 划分为 P_L 和 P_R
4. MinDistance(P_L, X_L, Y_L); $\delta_L = P_L$ 中的最小距离
5. MinDistance(P_R, X_R, Y_R); $\delta_R = P_R$ 中的最小距离
6. $\delta = \min(\delta_L, \delta_R)$
7. 对于在 l 线左边距离 δ 内每个点, 检查右边是否有与之距离小于 δ 的点, 如果存在则将 δ 修改为新值

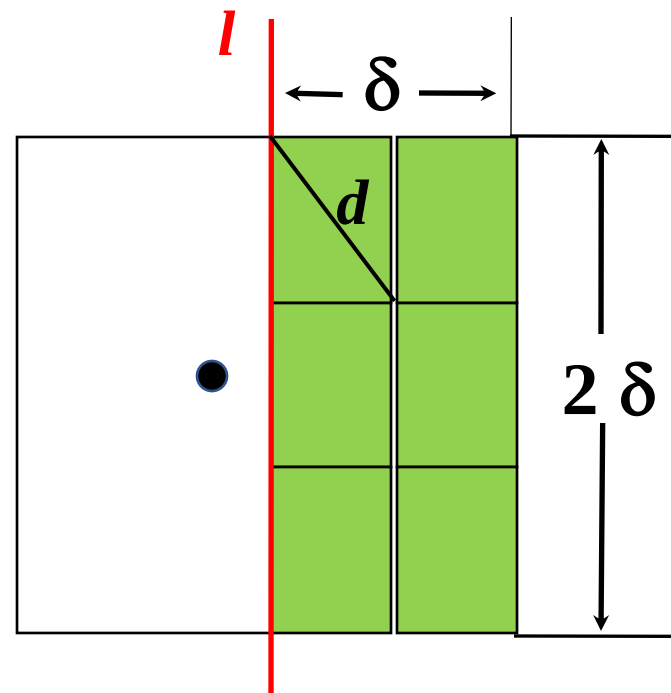
改进分治2

- 平面最近点划分实例
- 算法伪码
- 跨边界点处理
- 算法分析
- 增加预处理
- 实例分析
- 改进后复杂度

小结

跨边界处理

$$\begin{aligned}
 d &= \sqrt{(\delta/2)^2 + (2\delta/3)^2} \\
 &= \sqrt{\delta^2/4 + 4\delta^2/9} \\
 &= \sqrt{25\delta^2/36} = 5\delta/6
 \end{aligned}$$



右边每个小方格至多1个点，每个点至多比较对面的6个点，检查1个点是常数时间， $O(n)$ 个点需要 $O(n)$ 时间



改进分治2

平面最近点

划分实例

算法伪码

跨边界点处理

算法分析

增加预处理

实例分析

改进后复杂度

小结

算法分析

分析：

步1递归边界处理

$O(1)$

步2排序

$O(n \log n)$

步3划分

$O(1)$

步4-5子问题

$2T(n/2)$

步6 确定 δ

$O(1)$

步7检查跨边界点对

$O(n)$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n)$$

$$T(n) = O(1) \quad n \leq 3$$



还能有更好的方法么

由递归树估计 $T(n) = O(n \log^2 n)$



改进分治2

平面最近点

划分实例

算法伪码

跨边界点处理

算法分析

增加预处理

实例分析

改进后复杂度

小结

增加预处理

原算法：

在每次划分时对子问题数组重新排序

改进算法：

1. 在递归前对 X , Y 排序, 作为预处理

2. 划分时对排序的数组 X , Y 进行拆分, 得到针对子问题 P_L 的数组 X_L , Y_L 及针对子问题 P_R 的数组 X_R , Y_R

排序问题工作量减少了, 总问题工作量也减少了

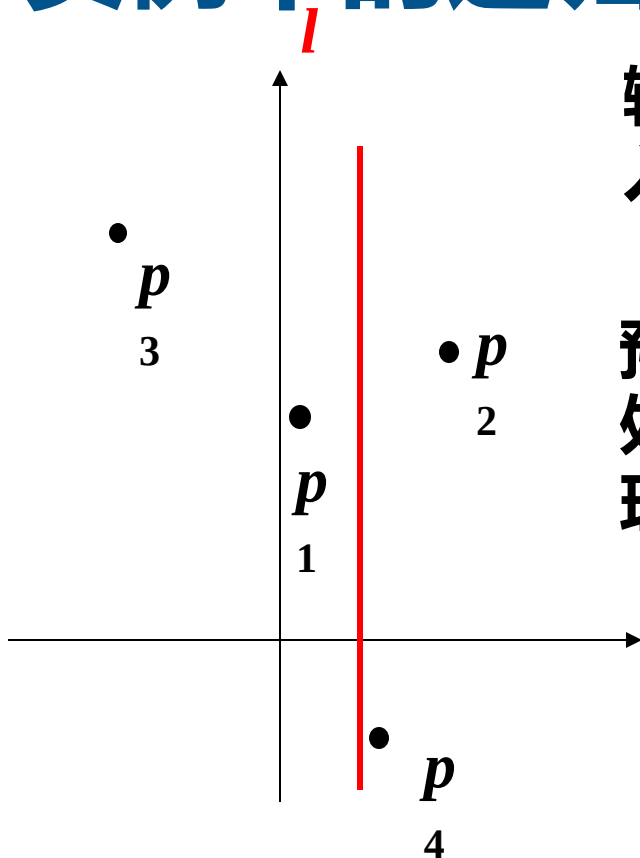
原问题规模为 n , 拆分的时间为 $O(n)$



改进分治2

- 平面最近点划分实例
- 算法伪码
- 跨边界点处理
- 算法分析
- 增加预处理
- 实例分析
- 改进后复杂度
- 小结

实例中的递归



输入

P	1	2	3	4
X	0.5	2	-2	1
Y	2	3	4	-1

预处理

X	-2(3)	0.5(1)	1(4)	2(2)
Y	-1(4)	2(1)	3(2)	4(3)

处理后

X_L	-2(3)	0.5(1)
X_R	1(4)	2(2)
Y_L	2(1)	4(3)
Y_R	-1(4)	3(2)

拆分的时间复杂度为 n



改进分治2

平面最近点
划分实例
算法伪码
跨边界点处理
算法分析
增加预处理
实例分析
改进后复杂度

小结

改进算法的时间复杂度

$W(n)$ 为算法时间复杂度

递归过程: $T(n)$,预处理 $O(n\log n)$

$$\begin{aligned} W(n) &= T(n) + O(n\log n) \\ T(n) &= 2T(n/2) + O(n) \\ T(n) &= O(1) \quad n \leq 3 \end{aligned}$$

解得 $T(n) = O(n\log n)$

于是 $W(n) = \underbrace{O(n\log n)}_{\text{排序时间}} + \underbrace{O(n\log n)}_{\text{递归时间}} = O(n\log n)$

原来的复杂度 $T(n) = O(n\log^2 n)$



小结

- 依据

$$W(n) = aW(n/b) + f(n)$$

- 提高算法效率的方法：

- -减少子问题个数 a ：

- $W(n) = O(n^{\log_b a})$

- -增加预处理，减少 $f(n)$

作业：自行选择一个算法问题，请以学术论文规范形式完成，报告以计算机学报模板完成，字数**1500+**，PPT格式不限，具体要求如下：

- 1.报告包括题目（自拟），摘要，引言，其中引言部分，需介绍问题的研究背景和研究意义，而不是脱离实际意义的单个实例；
- 2.对算法进行分析描述，比如给出算法的递归方程，诸如此类；
- 3.分析算法计算复杂度；
- 4.给出算法伪代码；
- 5.实验结果（运用你熟悉的代码，推荐以 Matlab、Python 给出数值算例，实验结果应尽量丰富，如果有代码，请放在附录）；
- 6.结论。